

Urban Heat in Zurich (1985-2024)

A. Introduction

The aim of this project is to identify and analyse patterns of temperature and vegetation in Zurich between the years 1985 and 2024, using Landsat data. For this, Land Surface Temperature (LST) and Normalized Difference Vegetation Index (NDVI), an index for identifying vegetation, were analysed.

The 2 core questions asked are:

1. *How did LST and NDVI change from 1985 to 2024?*
2. *How do NDVI and LST correlate and is this correlation affected by climate change?*

B. Workflow

The Analysis was carried out in a Jupyter Notebook, structured into 3 parts. These sections are numbered 0-2 and correspond to the chosen research questions. In this report, numbers that reference the code structure will be written in *italics*.

Part 0; Preparation

In this part, all necessary modules are imported, the data can be automatically downloaded from google drive and the relevant bands are imported into the notebook. The bands of each year, which are separate files, are combined into one data cube per year, with time as a coordinate. The NDVI is calculated from the NIR- and red-bands and saved as a separate object. To set the scene for the rest of the analysis, a true-colour RGB image is generated.

Part 1; How did LST and NDVI change from 1985 to 2024?

To start answering this question and get a feeling for overarching trends, simple visualisations of the two analysed variables were generated in 1.1 and 1.2. Using hvplot, an interactive side by side plot enables visual inspection of LST and NDVI throughout the years in 1.1, and, by calculating the difference of each year to 1985, a comparison to the oldest layer. Unfortunately, these animations could not be printed to pdf and can only be looked at by executing the notebook.

Next, change was modelled using simple linear regression. In section 1.3, change was modelled while collapsing the spatial dimension. For the LST this was done by taking the mean temperature. This makes less sense for the NDVI (while an NDVI of 0.2 is higher than an NDVI of 0.1, it's most likely still not vegetation). The aggregation function used was the sum of all pixels with a NDVI higher than 0.3, a common threshold for NDVI analysis.^[1] To give this number more meaning, it was then converted to km² using the spatial resolution.

To identify spatial patterns in the change, in section 1.4, the spatial properties of the data were kept for the analysis. A linear model was fitted to every pixel separately and the resulting slope visualized as a map.

Part 2; How do NDVI and LST correlate?

To answer this question, LST was plotted against NDVI. Again, a linear regression line was fitted to quantify the relationship. To not overload the plot, this was first done with one year in 2.1. In 2.3, the regression lines of all years were plotted to analyse differences in time.

C. Results and Interpretations

1. How did LST and NDVI change from 1985 to 2024?

Not surprisingly, the Land Surface Temperature in Zurich has increased steadily since 1985. In 1.3, the mean increase is estimated as $+1.46\text{ }^{\circ}\text{C decade}^{-1}$, which is very high compared to the current global estimate of $0.35\text{ }^{\circ}\text{C decade}^{-1}$.^[2] For this there are many explanations. Firstly, Switzerland is a region where the effects of climate change exceed the global rate.^[3] Additionally, urban areas are known to have a more rapid surface warming than rural areas.^[4] Lastly, this analysis only uses data from July 1st and is therefore blind for seasonal effects.

NDVI too has a positive slope in 1.3. However, the value of this slope, $0.6\text{ km}^2\text{ decade}^{-1}$, is very small in comparison to the overall area inspected ($<1\%$), which is approximately 67 km^2 . Such a small change cannot be supported by the small amount of data we have. It may be caused by a lot of reasons other than an increase in vegetation-covered area (e.g. change in the sensor quality, effect of precipitation in the different years).

However, it must be kept in mind that a large amount of the analysed area is not urban but woodland area, which are strongly protected in Switzerland and cannot easily be cleared,^[5] so change would not be expected in these areas. Additionally, the coarse pixel size of 30 m^2 might not be able to accurately detect small scale changes in Vegetation in the urban areas of Zurich, such as the disappearance of gardens or singular trees.

When looking at the spatial patterns of the NDVI slope in 1.4, one area shows high increase. It's the Irchelpark, which was created and planted in 1986, so right after the beginning of our data.

Modeled change in NDVI (1985-2024)

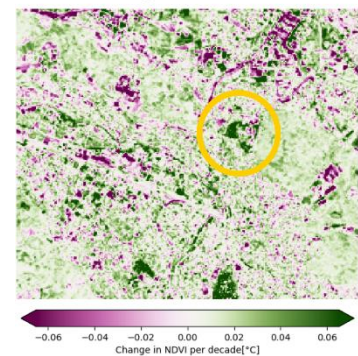


Figure 1 Output 1.4, Yellow: the region of the Campus Irchel

How do NDVI and LST correlate and is this correlation affected by climate change?

There is a clear and significant correlation between NDVI and LST ($R^2 = 0.63$). The slope is approximately $1.4\text{ }^{\circ}\text{C}$ per 0.1 change in NDVI. However, the relationship does not seem to be completely linear. The normal QQ-plot shows non-normal residuals in the lower quantiles. The other diagnostic plots show some patterning too. Even by visual examination of the data, we can see that there seems to be a nonlinear trend, especially at high NDVI where temperature drops fast, suggesting strong cooling effects in areas with dense vegetation.

By comparing the slope across all years, it seems that in Zurich, climate change does not have an impact on the slope between NDVI and LST, meaning areas in vegetation do not heat up slower than built-up areas. The analysis in R however detects a significant improvement by adding the interaction between year and NDVI ($p\text{-value} = 0.04$, $df = 49968$). However, because the pixels are not independent, which can lead to overconfident p -values, and the very small effect size ($<0.01\text{ }^{\circ}\text{C / year}$), this effect is most likely not relevant.

D. Challenges and Reflection

Overall, the analysis of the NDVI was more challenging. Unlike Temperature, the NDVI is not meaningful on itself but has to be interpreted first.

Another challenge was to identify useful and meaningful visualizations, because it is very easy to generate lots of different plot without gaining new insight. This resulted in a lot of code which was later discarded as redundant.

E. Appendix

Repository Link

Link to GitHub repository:

https://github.com/MinaKimBebi/Urban_Heat_Zurich_sds210_mbebi

Sources and AI

1. J. Walz, K. Weber; *Identifying active growing vegetation using NDVI Threshold values*, https://www.researchgate.net/publication/353828481_IDENTIFYING_ACTIVELY_GROWING_VEGETATION_USING_NDVI_THRESHOLD_VALUES, as at 13.05.2026
2. MeteoSchweiz; *Steigende Temperaturen*, <https://www.meteoschweiz.admin.ch/klima/klimawandel/steigende-temperaturen.html> as at 13.05.2026
3. A. Witze; *Climate change is speeding up — the pace nearly doubled in ten years*: <https://www.nature.com/articles/d41586-026-00745-z> as at 13.05.2026
4. Z. Liu et al. *Climate change is speeding up — the pace nearly doubled in ten years*, <https://www.nature.com/articles/s43247-022-00539-x#Sec2> as at 13.05.2026
5. *Bundesgesetz über den Wald*, https://www.fedlex.admin.ch/eli/cc/1992/2521_2521_2521/de as at 13.05.2026

AI has been used to in coding to find specific functions, parameters or find and understand syntax errors. It has further been used to understand the provided Google Earth Engine script and to detect spelling mistakes in this report. It has not been used to write, analyse or interpret any results and all final logic and writing was verified by me.

Notebook Printout:

0. Preparation

0.1 Load Modules & create folders

```
In [20]: import gdown
import os
import xarray as xr
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
import cmcramer.cm as cmc
import hvplot.xarray
```

create all necessary folders

```
In [21]: folder_raw = "../data/raw"
folder_processed = "../data/processed"
folder_outputs = "../outputs"

os.makedirs(folder_raw, exist_ok=True)
os.makedirs(folder_processed, exist_ok=True)
os.makedirs(folder_outputs, exist_ok=True)
```

0.2 Download data

the data was downloaded yousing the following *Google Earth Engine* script: <https://code.earthengine.google.com/04afb4de7a1d95d988d2972e3ed87ffd>. --> downloads interpolated image of the 1st of July 1st of each 3-year block

Band positions: 1. *Blue*, 2. *Green*, 3. *Red*, 4. *NIR*, 5. *SWIR1*, 6. *SWIR2*, 7. *TIR1*

Automatic download from Google Drive:

```
In [22]: # Dictionary of file IDs per year(always the first of 3-year block)
datasets = {
    1985: "1gdgTwrMUVVMpNC5YYTU3LkgNEutjRUM5",
    1988: "11YyrBD97drS3H_pxI0nLmXpU4S9Zw8b1",
    1991: "1JxT2CNw6y1RTHXwFJ0nm-_4unLJxBRUJ",
    1994: "1TOFy-yTz5KNsVAH-dh0YeJLSb8pAqFas",
    1997: "1ysdgB8XYwEEYX1578CMQzVK8ZZ2XHTku",
    2000: "1s7Nn180wGbMh1xyEV7Dx9YqaZPNpOZb_",
    2003: "11ES90yhetkfVH8DT0i5BCJdowVLCCQX7",
    2006: "1S-LoLT967qDS8d1bp_wXaRnja0JscuJu",
    2009: "1doUKH38hT1t1e41qeQ7AH8aDjGIQ0F90",
    2012: "1o4sFKySMZ11e2W9cGU3EyB4KC09CvJZh",
    2015: "1XwhUhv3FW4sFx3ubGf9v-dyq9I-Idq_b",
    2018: "1t16HUGh17ldTojX31rt3MMH_PA2R3kV4",
```

```

2021:"1qh7ZLA0w36l28qs8JJlaKs6FzcjmLigY",
2024:"1mc0cWZgh6rm0YfZuPlIm8FWJKvMfsDQc"
}

#create array containing all years
year_list = np.arange(1985, 2025, 3)

#loop through them an download missing files
for year in year_list:
    filepath = f"../data/raw/LandsatComposite_Zurich_{year}.tif"
    fileID = datasets[year]
    if not os.path.exists(filepath):
        print(f"Downloading {filepath}...")
        url = f"https://drive.google.com/uc?id={fileID}"
        gdown.download(url, filepath, quiet=False)
    else:
        print(f"{filepath} already exists")

```

```

../data/raw/LandsatComposite_Zurich_1985.tif already exists
../data/raw/LandsatComposite_Zurich_1988.tif already exists
../data/raw/LandsatComposite_Zurich_1991.tif already exists
../data/raw/LandsatComposite_Zurich_1994.tif already exists
../data/raw/LandsatComposite_Zurich_1997.tif already exists
../data/raw/LandsatComposite_Zurich_2000.tif already exists
../data/raw/LandsatComposite_Zurich_2003.tif already exists
../data/raw/LandsatComposite_Zurich_2006.tif already exists
../data/raw/LandsatComposite_Zurich_2009.tif already exists
../data/raw/LandsatComposite_Zurich_2012.tif already exists
../data/raw/LandsatComposite_Zurich_2015.tif already exists
../data/raw/LandsatComposite_Zurich_2018.tif already exists
../data/raw/LandsatComposite_Zurich_2021.tif already exists
../data/raw/LandsatComposite_Zurich_2024.tif already exists

```

0.3 Load Data

```

In [23]: #create empty arrays to store data
LST_array = []
Red_array = []
NIR_array = []

#add data from every year (seperat files)
for year in year_list:
    filepath = f"../data/raw/LandsatComposite_Zurich_{year}.tif"
    with xr.open_dataset(filepath, engine = "rasterio") as src: # only open nee
        src_t = src.expand_dims(time=[year])# add time as a coordinate
        LST_array.append(src_t.sel(band = 7).band_data)
        Red_array.append(src_t.sel(band = 3).band_data)
        NIR_array.append(src_t.sel(band = 4).band_data)

# turn into xr-array
Red = xr.concat(Red_array, dim = "time")
NIR = xr.concat(NIR_array, dim = "time")
LST = xr.concat(LST_array, dim="time")

```

0.4 calculate °C and NDVI

To make the values of LST more intuitive, they are converted to °C The NDVI is calculated. A filtered version is made for analysis where we only want to look at what is classified as vegetation (e.g 1.1 and 1.3)

```
In [24]: # change LST unit from K into °C
LST = LST - 273.15

#calculate raw NDVI
NDVI = (NIR - Red)/(NIR + Red)
#set overly large values to 1 ()
NDVI = NDVI.clip(min=-1, max=1)

# produce thresholded layer
THRESHOLD = 0.3 # standart for mixed vegetation
NDVI_filtered = NDVI.where(NDVI >= THRESHOLD)
```

0.5 RGB picture for reference

This codeblock generates a true color image to give contex to the following analysis

```
In [25]: def normalize(array, vmin=0, vmax=0.2):
    """Normalize and clip array to a specific range (default 0 to 0.4)"""
    # Clip the array to the specified vmin and vmax
    clipped = np.clip(array, vmin, vmax)
    # Scale the clipped array to 0-1 for display
    return (clipped - vmin) / (vmax - vmin)

#select a year
reference_year = 2024
if not reference_year in year_list:#check if selected year is valid
    print(f"{reference_year} is not an available file")

else:
    #extract rgb bands from reference year
    with xr.open_dataset(f"../data/raw/LandsatComposite_Zurich_{reference_year}.tif")
        # load bands and get rid of outliers for effective visualization
        Blue_ref = normalize(src.sel(band = 1).band_data)
        Green_ref = normalize(src.sel(band = 2).band_data)
        Red_ref = normalize(src.sel(band=3).band_data)

    #stack the bands
    rgb = np.dstack((Red_ref, Green_ref, Blue_ref))

    fig, ax = plt.subplots(figsize = (8,4))
    ax.imshow(rgb)
    ax.set_title(f"Analyzed area {reference_year} (truecolor)", fontsize = 15)
    ax.set_axis_off()

    fig.savefig(f"../outputs/0_5_reference_picture_RGB")
```

Analyzed area 2024 (truecolor)



1. How did LST and NDVI change from 1985 to 2024?

1.1 Simple Visualization of the Variabels

This code produces interactive hvplot animations of LST and NDVI in the different years

```
In [26]: def limits(array, p = 2):
    """returns upper and lower perectile of an array
    (default: vmin = 2nd percentile, vmax = 98th percentile)"""
    vmin = np.percentile(array, p)
    vmax = np.percentile(array, 100-p)
    return vmin, vmax

#plot LST
LST_min_hv, LST_max_hv = limits(LST)

LST_hvplot = LST.hvplot(x = "x", y = "y", groupby = "time", cmap = cmc.lajolla_r,
    clim = (LST_min_hv, LST_max_hv),
    clabel = "Land Surface Temperature [°C]",
    aspect='equal',
    title = "LST in Zurich (1985-2024)",
    fontsize = 15,
    xaxis = None, yaxis = None)

NDVI_filtered = NDVI.where(NDVI >= THRESHOLD)
#plot NDVI
NDVI_min_hv, NDVI_max_hv = limits(NDVI_filtered)
#NDVI_max_hv=np.percentile(NDVI, 98) #only max used -->almost no nega
NDVI_hvplot = NDVI_filtered.hvplot(x = "x", y = "y", groupby = "time", cmap = "YlGn
    clim = (THRESHOLD, 1),# 1 = normal upper limit of NDVI
```



```

        clabel = "NDVI",
        aspect='equal',
        title = "NDVI in Zurich (1985-2024)",
        fontsize = 15,
        shared_axes = False,
        xaxis = None, yaxis = None,
        bgcolor="lightgray")

```

LST_hvplot + NDVI_hvplot

Out[26]: BokehModel(combine_events=True, render_bundle={'docs_json': {'8f70f229-9271-47ea-b577-c12c29810d04': {'version...

Visual interpretation:

LST: definite increase

NDVI: no clear trend, a lot of fluctuations

1.2 NDVI change in reference to 1985

This code calculates the difference of each layer to the reference year creates an animation through time

```

In [27]: #calculate difference
LST_dif = LST.sel(time = slice(1988,2024)) - LST.sel(time = 1985)

#set boundaries for visualisation centered around 0
LST_min, LST_max = limits(LST_dif)
LST_boundry = max(abs(LST_min), abs(LST_max))# center around 0

#animate LST
LSTdif = LST_dif.hvplot(x = "x", y = "y", groupby = "time", cmap = cmc.vik,
                        clim = (-LST_boundry, LST_boundry),
                        clabel = "Change in LST in reference to 1985",
                        aspect='equal',
                        title = "Change in LST in respect to 1985",
                        fontsize = 15,
                        shared_axes = False,
                        xaxis = None, yaxis = None)

#calculate difference
NDVI_dif = NDVI.sel(time = slice(1988,2024)) - NDVI.sel(time = 1985)

#set boundaries for visualisation centered around 0
NDVI_min, NDVI_max = limits(NDVI_dif)
NDVI_boundry = max(abs(NDVI_min), abs(NDVI_max))# center around 0

#animate NDVI
NDVIDif = NDVI_dif.hvplot(x = "x", y = "y", groupby = "time", cmap = cmc.bam,
                          clim = (-NDVI_boundry, NDVI_boundry),
                          clabel = "Change in NDVI in reference to 1985",
                          aspect='equal',

```



```

        title = "Change in NDVI in respect to 1985",
        fontsize = 15,
        shared_axes = False,
        xaxis = None, yaxis = None)

#show plots
LSTdif + NDVIDif

```

Out[27]: BokehModel(combine_events=True, render_bundle={'docs_json': {'06b0a427-4fe0-4457-a550-544293424aad': {'version...

Visual interpretation:

LST: Some cooler years in the beginning, in the end almost the entire area is warmer then it was in 1985

NDVI: Overall there seems to be an increase in change intensities in both directions. Because we can also see change in areas that were forest through out the whole time, we can however not rule out that some of these effects are random or due to changes in sensor sensibility.

1.3 Modelling change in Collapsed space

In this Segment, the spatial extent of the data is collapsed, resulting in one overall value per 3-annual period. To these values, a linear model is fitted to measure the rate of change. For LST, the aggregating function used is the mean. For NDVI, the number of pixels that exceed the THRESHOLD are counted and converted to km^2

```

In [28]: # change in overall LST
LST_mean = LST.mean(dim=["x","y"])
LST_mean_fit = LST_mean.polyfit(dim = "time", deg = 1)
LST_mean_trend_line = xr.polyval(LST_mean.time, LST_mean_fit.polyfit_coefficients)

# change in overall NDVI
pixelsize = NDVI.rio.resolution()
NDVI_sum = NDVI_filtered.sum(dim=["x","y"])*(pixelsize[0]**2)/ 1000000## to get km

NDVI_sum_fit = NDVI_sum.polyfit(dim = "time", deg = 1)
NDVI_sum_trend_line = xr.polyval(NDVI_sum.time, NDVI_sum_fit.polyfit_coefficients)

```

The calculated slope is plotted with the data for both datasets

```

In [29]: fig, ax = plt.subplots(1,2, figsize = (16,8))

LST_mean.plot(ax = ax[0],marker = "o", ls= "", label = "sampled LST", color = "lightblue")
LST_mean_trend_line.plot(ax = ax[0], label = "modelled LST", color = "tomato")
ax[0].set_title("Mean LST of Zurich (1985-2024)",fontsize = 20)
ax[0].legend()
ax[0].set_ylabel("LST [°C]")
ax[0].set_xlabel("year")
ax[0].annotate(f"Estimate + {LST_mean_fit.polyfit_coefficients[0]*10:.2f} °C/10yrs"

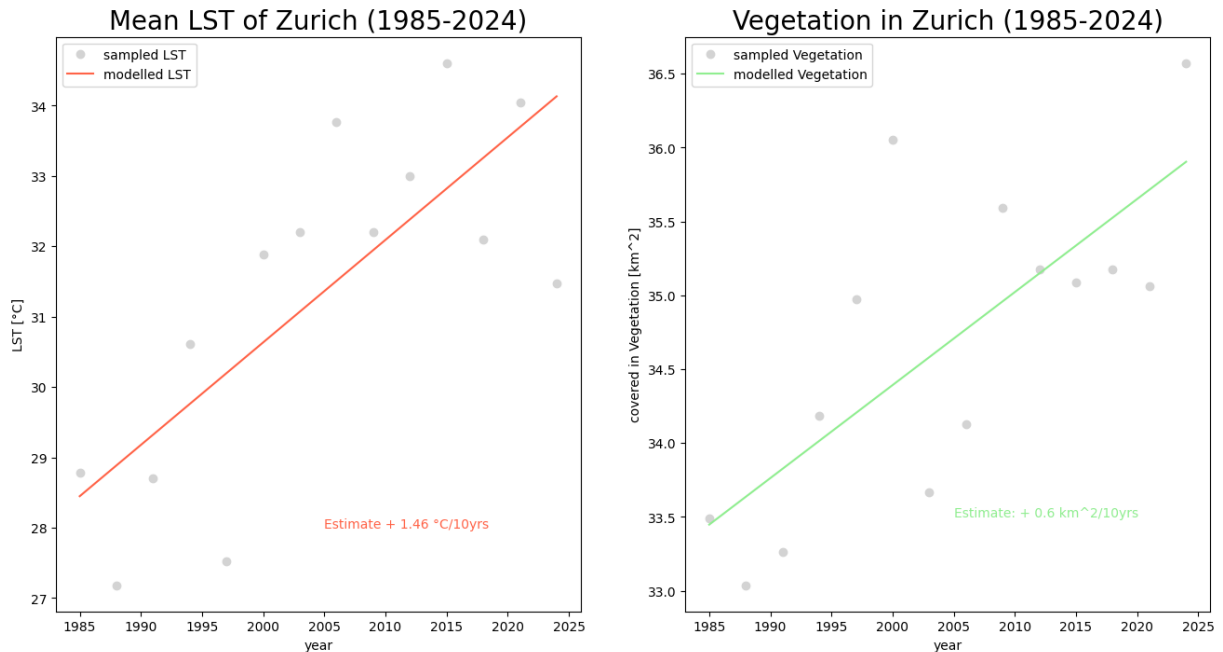
```

```

NDVI_sum.plot(ax = ax[1], marker = "o", ls = "", label = "sampled Vegetation", color = "lightgray")
NDVI_sum_trend_line.plot(ax= ax[1], label = "modelled Vegetation", color = "lightgreen")
ax[1].set_title("Vegetation in Zurich (1985-2024)", fontsize = 20)
ax[1].legend()
ax[1].set_ylabel("covered in Vegetation [km^2]")
ax[1].set_xlabel("year")
ax[1].annotate(f"Estimate: + {(NDVI_sum_fit.polyfit_coefficients[0]*10):.1f} km^2/10yrs")

fig.savefig(f"../outputs/1_3_Modelling_collapsed_space", bbox_inches = "tight")

```



Short Interpretation (for more details see the provided report):

LST: Like the visual inspections, this plot shows a increase of ca. 1.46 °C per decade, a very strong increase

NDVI: Because of the plot sclaing, there seems to be a big effect. Upon closer inspection of the slope we can see that the increase is only 0.6 km² per decade, which is very small in comparison to the whole analysed area (67km²).

1.4 Modelling Change pixelwise

Here, similarly to 2.3, the rate of change is measured using a linear regression, but this time a regrssion is fitted for every pixel seperatly. For easier interpretation, the slope values are changed from annual rates to per decade values.

```

In [30]: # change in LST per pixel
LST_fit = LST.polyfit(dim = "time", deg = 1)
LST_slope10 = LST_fit.polyfit_coefficients.sel(degree = 1)*10

#change in NDVI per pixel
NDVI_fit = NDVI.polyfit(dim = "time", deg = 1)

```

```
NDVI_slope10 = NDVI_fit.polyfit_coefficients.sel(degree = 1)*10
```

The slope of every pixel is mapped onto separate maps for LST and NDVI

```
In [31]: fig, ax = plt.subplots(1,2, figsize = (16,8))

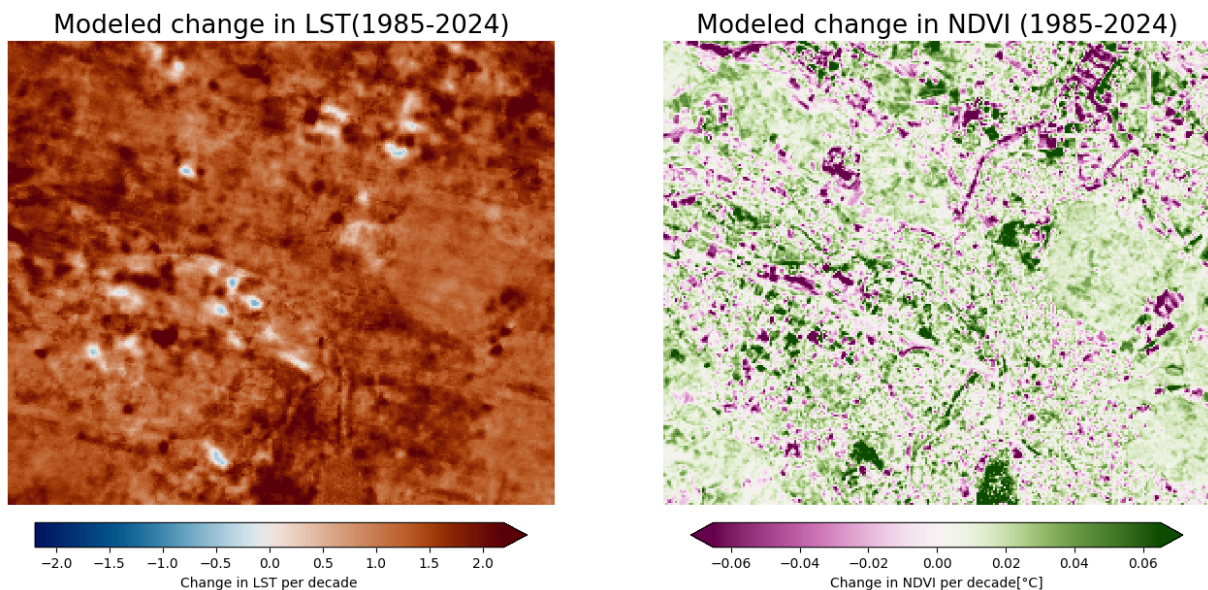
# plot slope of LST
LST_slope_min, LST_slope_max = limits(LST_slope10)
LST_slope_boundry = max(abs(LST_slope_min), abs(LST_slope_max))

LST_slope10.plot(ax=ax[0],
                 vmin = -LST_slope_boundry,
                 vmax = LST_slope_boundry,
                 cmap = cmc.vik,
                 cbar_kwargs = {"label" : "Change in LST per decade", "shrink" : 0.
ax[0].set_axis_off()
ax[0].set_title("Modeled change in LST(1985-2024)", fontsize = 19)
ax[0].set_aspect('equal')

#plot slope of NDVI
NDVI_slope_min, NDVI_slope_max = limits(NDVI_slope10)
NDVI__slope_boundry = max(abs(NDVI_slope_min), abs(NDVI_slope_max))

NDVI_slope10.plot(ax= ax[1],
                 vmin = - NDVI__slope_boundry,
                 vmax = NDVI__slope_boundry,
                 cmap = cmc.bam,
                 cbar_kwargs = {"label" : "Change in NDVI per decade[°C]", "shrink
ax[1].set_axis_off()
ax[1].set_title("Modeled change in NDVI (1985-2024)", fontsize = 19)
ax[1].set_aspect('equal')

fig.savefig(f"../outputs/1_4_Modelling_pixelwise", bbox_inches = "tight")
```



Visual interpretation:

LST: there are some regions where the model predicts an increase in temperature. However,

the vast majority of Zurich has however seen increasing temperatures

NDVI: We see that there is an increase in urban as well as in forest areas. They are however very small (<0.1), further suggesting that a big part of them is due to random fluctuations. There are however some clear patterns: We can see for example the Irchel park.

2. How do NDVI und LST correlate and is this correlation effected by climate change?

2.1. How do NDVI und LST correlate?

Here, we fit a linear regression with LST as the response and NDVI as the explainer variable. To better see the relation between the 2 variables, we only look at one year, otherwise the effect of climate change masks the patterns.

```
In [32]: #extract data
YEAR = 2024 ## select a year
NDVI_y = NDVI.sel(time = YEAR, method = "nearest")
LST_y = LST.sel(time=YEAR, method = "nearest")

# bind NDVI and LST into one np array to use np.polyfit later
combined = np.column_stack((NDVI_y.values.flatten(), LST_y.values.flatten()))
## filter out lake/waterbodies from analysis (large leverage)
comb_filtered = combined[combined[:,0] >= 0]

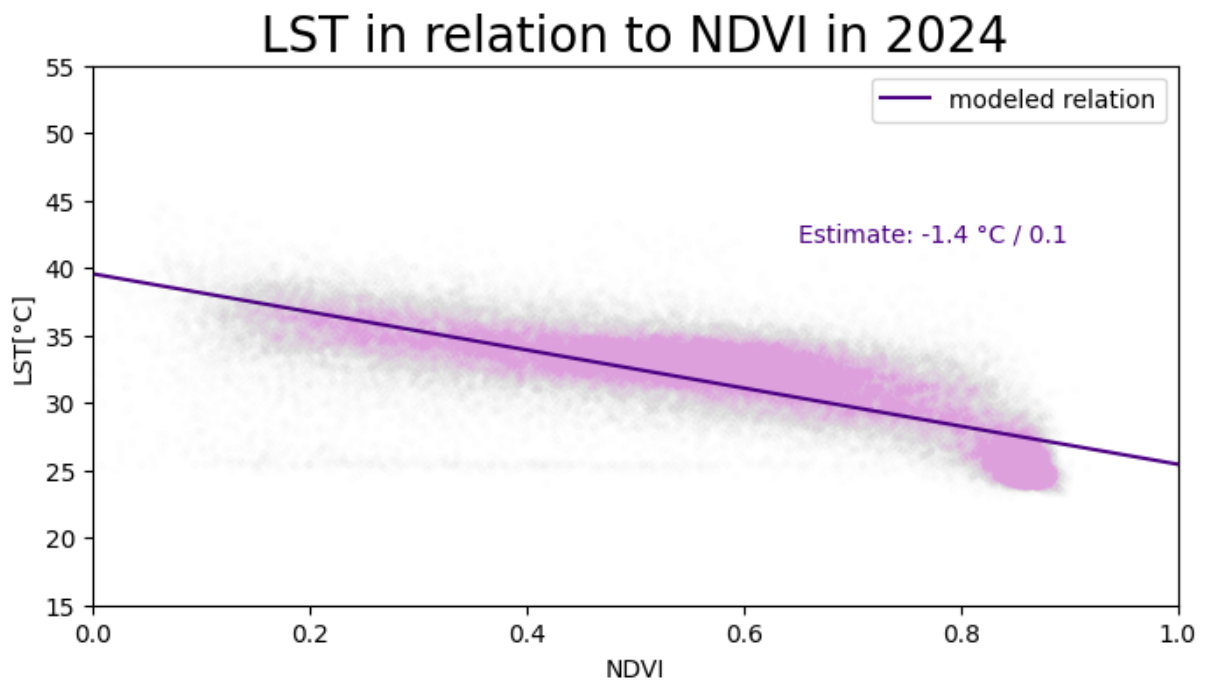
# calculate correlation
LST_NDVI_fit = np.polyfit(comb_filtered[:,0], comb_filtered[:,1], deg = 1)
intercept = LST_NDVI_fit[1]
slope = LST_NDVI_fit[0]
```

Plotting of LST against NDVI. Every Pixel is plotted with a very low alpha, so we can see where most points are more easily. The calculated regression line is then plotted on top

```
In [33]: fig, ax = plt.subplots(figsize = (8,4))

ax.scatter(comb_filtered[:,0], comb_filtered[:,1], marker = ".", alpha = 0.005, c =
ax.axline((0,intercept),slope=slope, color = "indigo", label = "modeled relation")
ax.set_xlim(xmin = 0, xmax= 1)
ax.set_ylim(15, 55)
ax.set_title(f"LST in relation to NDVI in {YEAR}", fontsize = 20)
ax.set_xlabel("NDVI")
ax.set_ylabel("LST[°C]")
ax.legend()
ax.annotate(f"Estimate: {slope * 0.1:.1f} °C / 0.1", (0.65,42), color = "indigo")

fig.savefig(f"../outputs/2_1_LST_to_NDVI_{YEAR}", bbox_inches = "tight")
```



Visual Interpretation:

There is a clear correlation between LST and NDVI. High NDVI seems to have an even larger effect than expected by linear regression.

For the analysis of the significance see R code.

2.2 Does the Correlation change with the advance of climate change?

In this section, the regression line is calculated for every year separately.

```
In [34]: #generate slope and intercept for every year
year_fits = []
for i, year in enumerate(NDVI.time.values):
    NDVI_y = NDVI.sel(time = year)
    LST_y = LST.sel(time = year)
    LST_NDVI_y = np.polyfit(NDVI_y.values.flatten(), LST_y.values.flatten(), deg = 1)
    intercept_y = LST_NDVI_y[1]
    slope_y = LST_NDVI_y[0]

    year_fits.append({"intercept": intercept_y,
                      "slope": slope_y,
                      "color": i,
                      "label": f"{year}"})
```

The slopes are then added one after another to the plot to get a feeling for the change in time

defining the update function:

```
In [35]: #define function on what changes in animation with each frame
def update(frame):
    """takes the frame and draws the corresponding line, updates the label to fit th
    #get year specific data
    fit = year_fits[frame]

    #draw line
    line = ax.axline(
        (0, fit["intercept"]),
        slope=fit["slope"],
        color=colors[fit["color"]],
        linewidth=2)

    #update year annotation
    time.set_text(fit["label"])
    return line, time
```

plotting:

```
In [36]: fig, ax = plt.subplots(figsize = (8,4))

# adding year of annotation to ax
time = ax.annotate("", (0.8,45), fontsize = 20)

# defines 1 color for every year to make a gradient fro yellow to red
colors = ['#ffffcc', '#fff2a8', '#fee187', '#fdc766', '#fdae4b', '#fd943f', '#fb7a3',
          '#f55c2e', '#ec4027', '#dc2a22', '#c5141f', '#ab071d', '#8e001b', '#700

#create animation
ani = FuncAnimation(
    fig,
    update,# the function defined in the block above
    frames=len(year_fits),
    interval=800)

#configure ax
ax.set_xlim(xmin = 0, xmax= 1)
ax.set_ylim(15, 55)
ax.set_title("Change in Relationship between NDVI and LST", fontsize = 19)
ax.set_xlabel("NDVI")
ax.set_ylabel("LST[°C]")

###fig.savefig(f"..../outputs/2_2_NDVI_LST_all_years") leere fig
display(HTML(ani.to_jshtml()))
```


the calculations without improving the accuracy of the result, a sample is taken.

```
In [37]: SAMPLE_SIZE = 50000
np.random.seed(11) #

# convert dataarrays to datasets and change the variable name to merge them into one
ds_NDVI = NDVI.to_dataset()
ds_NDVI = ds_NDVI.rename({"band_data": "NDVI"})

ds_LST = LST.to_dataset()
ds_LST = ds_LST.rename({"band_data": "LST"})

NDVI_LST = xr.merge([ds_NDVI, ds_LST], compat = "no_conflicts")
NDVI_LST.drop_vars(["spatial_ref", "band"])

#change them to dataframe for sampling & writing to csv
NDVI_LST = NDVI_LST.to_dataframe()
NDVI_LST_sample = NDVI_LST.sample(SAMPLE_SIZE)
NDVI_LST_sample.to_csv("../data/processed/NDVI_LST.csv")
```

Next Step:

Run the R script from top to bottom

```

#if not already done, set your working directory to the project folder
#setwd("xxx/Urban_Heat_Zurich_sds210_mbebi")
#load required libraries
library(tidyverse)
library(ggfortify)

#load data
NL = read_csv("data/processed/NDVI_LST.csv")

## exclude waterbodies from analysis:
filtered = filter(NL, NDVI >= 0)

# Analysis of significance for 2.1
## fit a simple linear model
fit = lm(LST ~ NDVI, data = filtered)

## control model assumptions
png(file="outputs/2_3_model_assumptions_simple_model.png")
autoplot(fit)
dev.off()
### we see some deviations from normal behaviour, which is not unexpected,
### because the pixels are not independent (common with spatial data)
### p-values should therefore be treated cautiously

## assess model fit
fit_summary = summary(fit)
fit_assessment = anova(fit)

print(paste("Coefficients: intercept:", round(fit$coefficients[1],1), "°C, slope:",
round(fit$coefficients[2],1), "°C/ NDVI unit"))
print(paste("NDVI explains", round(fit_summary$r.squared * 100, 2), "% of the variation
in LST"))
print(paste("the p-value is:", fit_assessment["NDVI", "Pr(>F)"]))# not trustworthy!

# Analysis of significance of explainer "year" for 2.2
#center time
centered = filtered |>mutate(time_cent = time - mean(time))

## fit a linear model, with the year and interaction included
fit_year = lm(LST ~ NDVI * time, data = centered)

## control model assumptions
png(file="outputs/2_3_model_assumptions_with_time.png")
autoplot(fit_year)# similar patterns as in analysis for 2.1
dev.off()

## assess model fit
fit_year_summary = summary(fit_year)
fit_year_assessment = anova(fit_year)

print(paste("the p-value for the interaction is:",
round(fit_year_assessment["NDVI:time", "Pr(>F)"], 2)))
print(paste("effect size of the interaction:", round(coef(fit_year)["NDVI:time"],
2),"°C / year"))
### there is significant interaction (but again p-value is to be used cautiously),
### --> time doesn't have the same effect at every NDVI value,
### the effect is however very small (a shift of 0.01 in the slope).

```